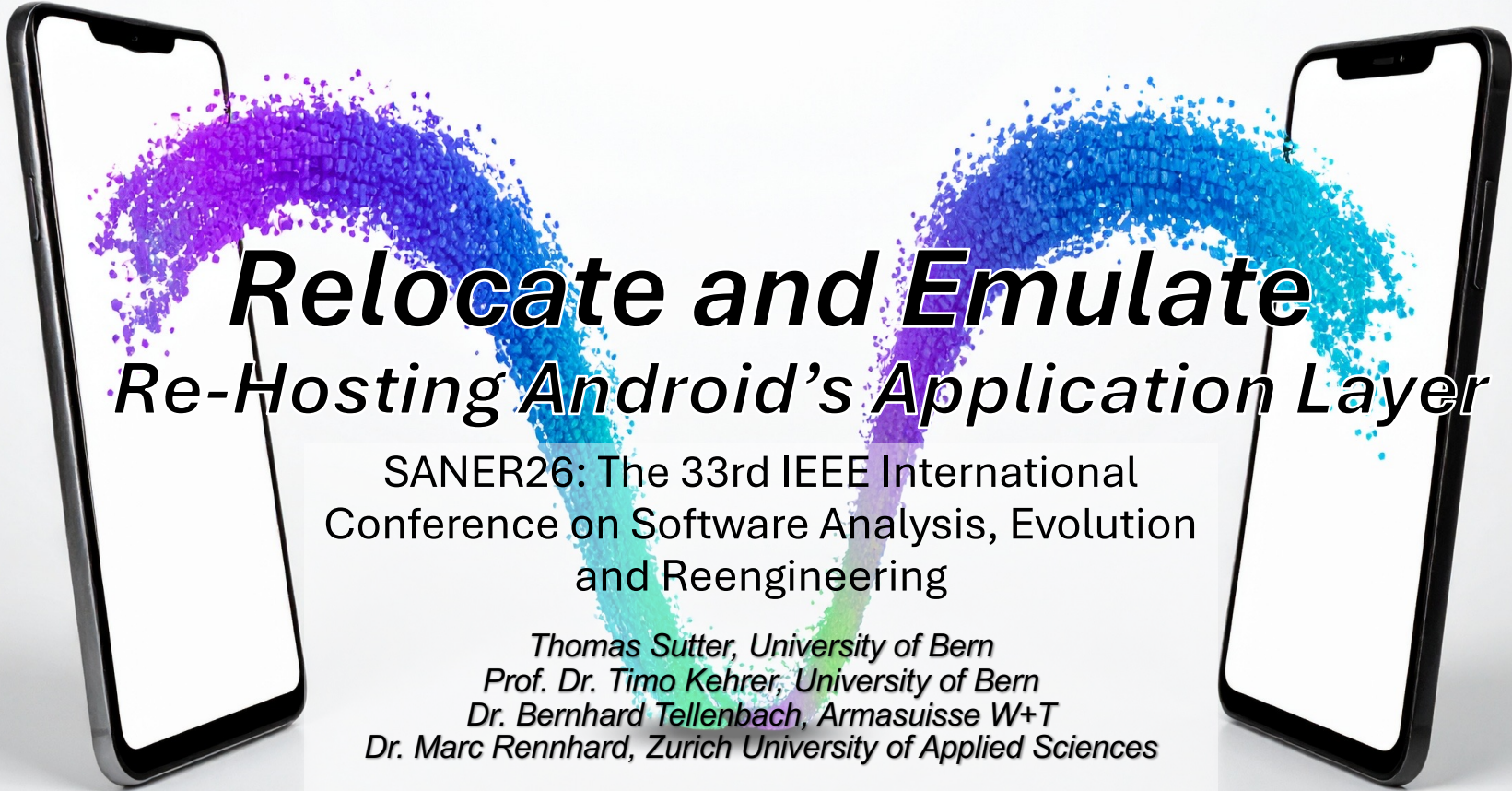




Relocate and Emulate ***Re-Hosting Android's Application Layer***

SANER26: The 33rd IEEE International
Conference on Software Analysis, Evolution
and Reengineering

Thomas Sutter, University of Bern
Prof. Dr. Timo Kehler, University of Bern
Dr. Bernhard Tellenbach, Armasuisse W+T
Dr. Marc Rennhard, Zurich University of Applied Sciences

Androids are everywhere...



TV-Boxes



Smartphones



Tablets



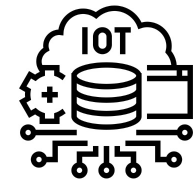
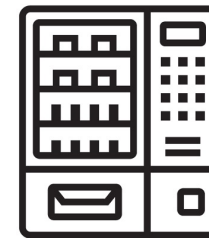
Car Infotainment System



Point-of-Sales Systems

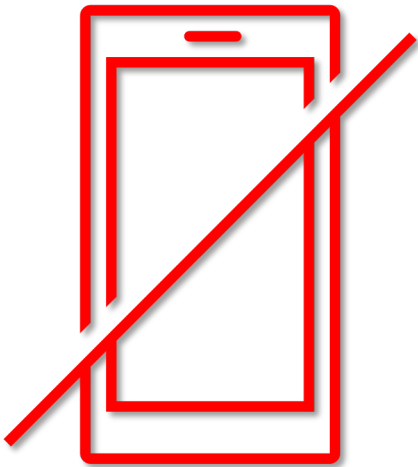


Wearables



Embedded & IoT Devices

Objectives



Problems → Physical Device Necessity, Fragmentation, Closed-Source Ecosystem, Platform constraints, etc.

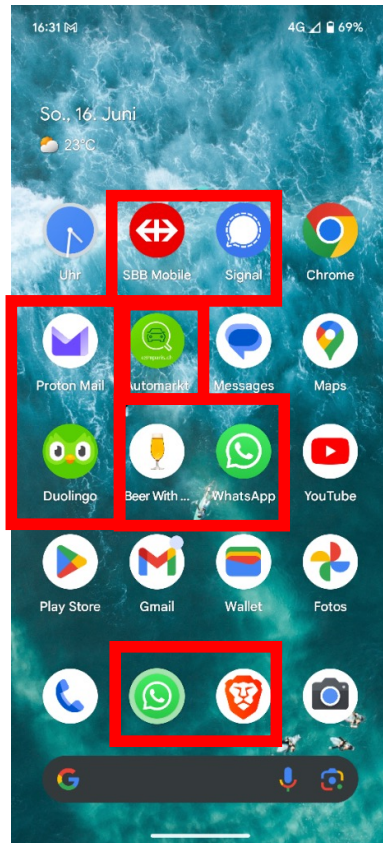
Idea → Re-Hosting the application layer to the Android Emulator

Goal → Developing a method to re-host real world Android device software onto the Android Emulator

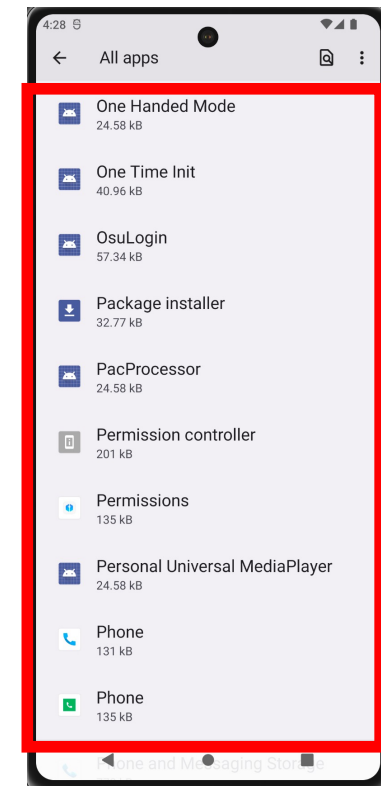
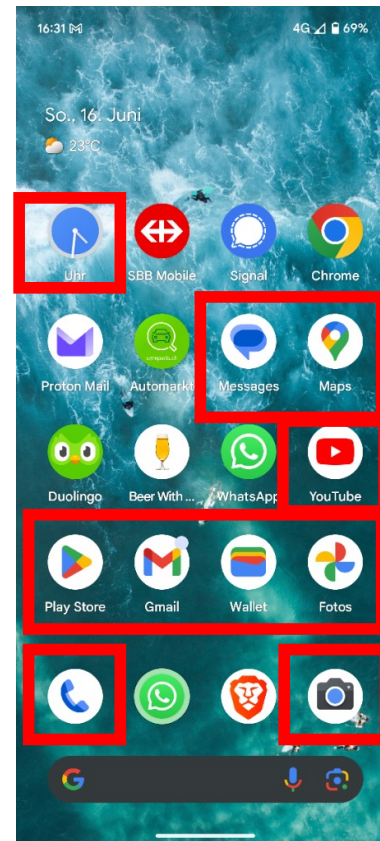
Purpose → Runtime Testing real-world code and vendor-customizations. Mainly, pre-installed Android apps.

Different types of apps

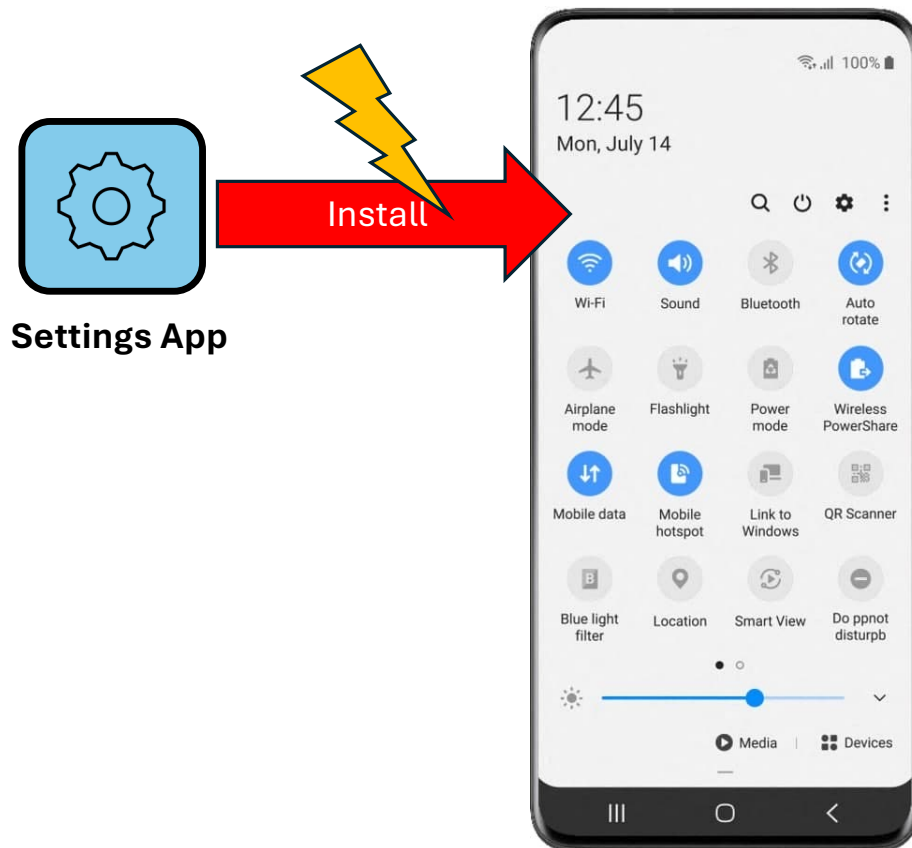
Userspace Apps



Pre-installed Apps



Challenges in testing pre-installed Apps



Platform Constraints

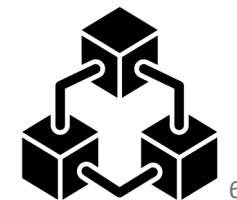
- ☐ System Partition Integration
- ☐ Singleton Apps
- ☐ Framework Dependencies

Security Constraints

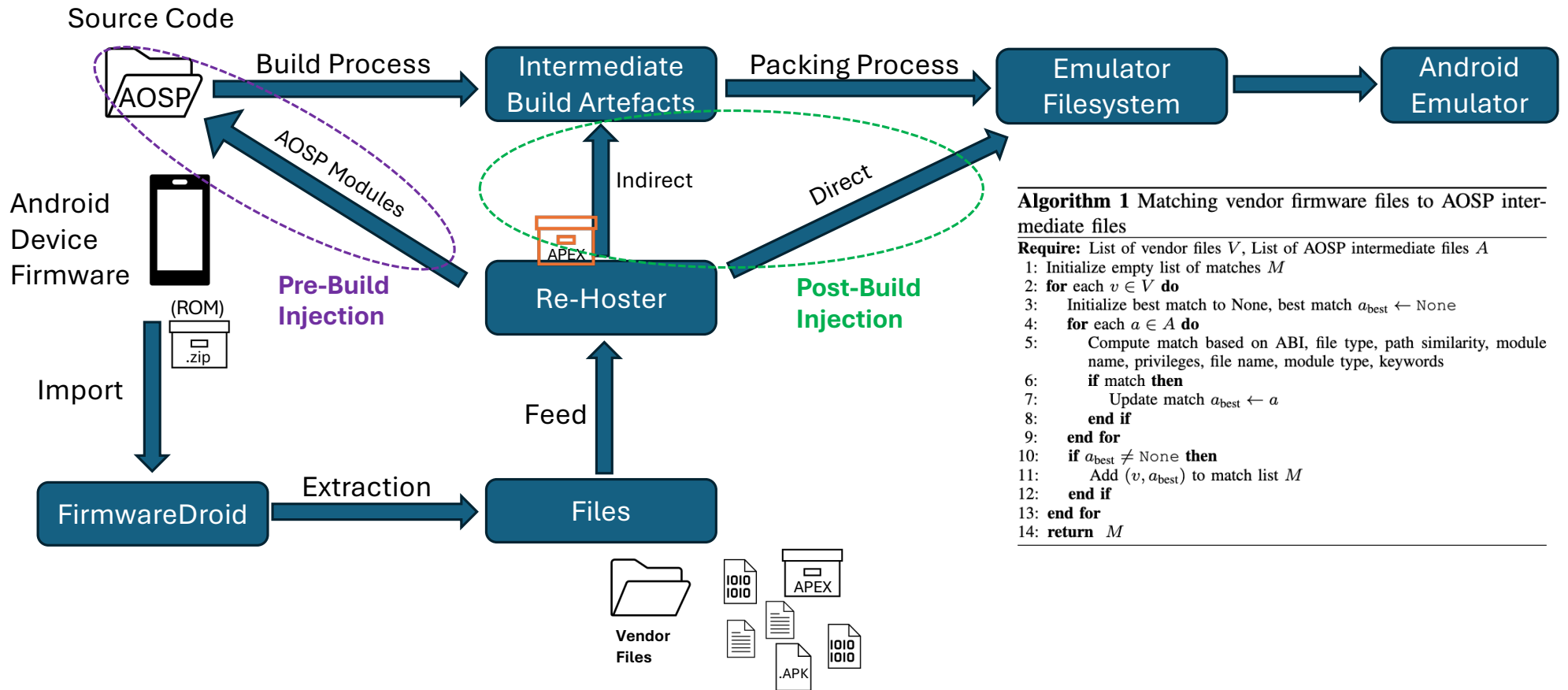
- ☐ Android Verified Boot
- ☐ Read-Only File Systems
- ☐ SELinux Policies
- ☐ Integrity Protections (Signatures)
- ☐ Permission Whitelisting
- ☐ Device Attestation
- ☐ No Root Rights on Device

App Constraints

- ☐ Inter-App Dependencies
- ☐ File and Native-Library Dependencies
- ☐ External Service Dependencies
- ☐ Environment Checks & Anti-Analysis



Our Re-Hosting Approach



Algorithm 1 Matching vendor firmware files to AOSP intermediate files

Require: List of vendor files V , List of AOSP intermediate files A

```

1: Initialize empty list of matches  $M$ 
2: for each  $v \in V$  do
3:   Initialize best match to None, best match  $a_{\text{best}} \leftarrow \text{None}$ 
4:   for each  $a \in A$  do
5:     Compute match based on ABI, file type, path similarity, module
       name, privileges, file name, module type, keywords
6:     if match then
7:       Update match  $a_{\text{best}} \leftarrow a$ 
8:     end if
9:   end for
10:  if  $a_{\text{best}} \neq \text{None}$  then
11:    Add  $(v, a_{\text{best}})$  to match list  $M$ 
12:  end if
13: end for
14: return  $M$ 
  
```

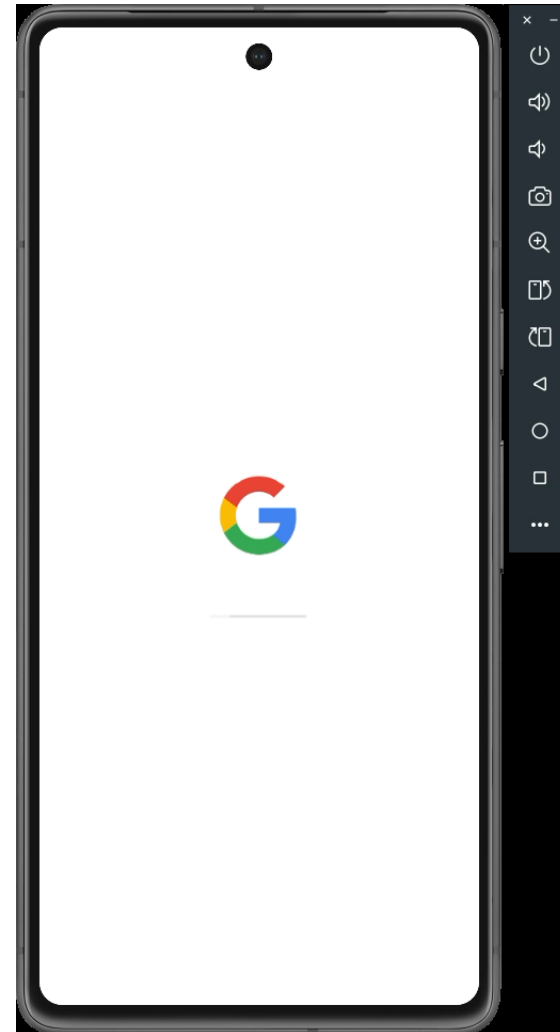
Demo

Re-Hosted Google Device:

- Including Core Services:
 - Android Runtime
 - Google Play Services
 - Selected Pre-Installed Apps

Showcasing:

- Starting Pixel Launcher
- Starting Settings App
- App Attestation (e.g., Gboard, Google Camera)
- Device Attestation (e.g., Google Play Protect)



Evaluation Setup

Dataset:

- 184 Android Smartphone Samples (Firmware / ROM)
- Testing different Android versions:
 - Android 12 – SDK 31
 - Android 12.1 – SDK 32
 - Android 13 – SDK 33

Baseline Injection Strategy:

- Generic non-vendor specific injection rules (conservative approach)

Metrics:

- Correctness: Success / Failure rates of different components (Build | Boot | Core | Launcher)
- File Coverage: File Injection Ratio
- Performance: Pre- and Post-Injection Overhead

Correctness

How good does the re-hosting works in practice?

- Successfully re-hosted 35% of the 184 samples
 - Baseline Injection Strategy works
 - Tradeoff stability vs. coverage
 - Improvements possible

CORRECTNESS OF RE-HOSTING: TASK SUCCESS WITHIN THE EMULATOR.

		Task Success				
Subset	#Sample	Build	Boot	Core	Launch	Avg. $S(\mathcal{D}_i)$
$\mathcal{D}_{33}$	73	1.0	1.0	0.41	0.41	0.70
$\mathcal{D}_{32}$	25	1.0	1.0	0.68	0.68	0.80
$\mathcal{D}_{31}$	86	0.9	0.9	0.18	0.18	0.54
Overall (Avg.)	184	0.97	0.97	0.42	0.42	0.68
Overall (Wtd. Avg.)	184	0.96	0.96	0.35	0.35	0.65

Our Re-Hosting method works and runtime testing is feasible. The baseline strategy has a moderate success rate overall and can be improved.

Correctness – Failure Analysis

Where does it frequently fail?

- Main failures during runtime of core services:
 - Missing files (native libraries)
 - Incompatible ABI (ARMv7)
 - ...

CORRECTNESS OF RE-HOSTING: TASK SUCCESS WITHIN THE EMULATOR.

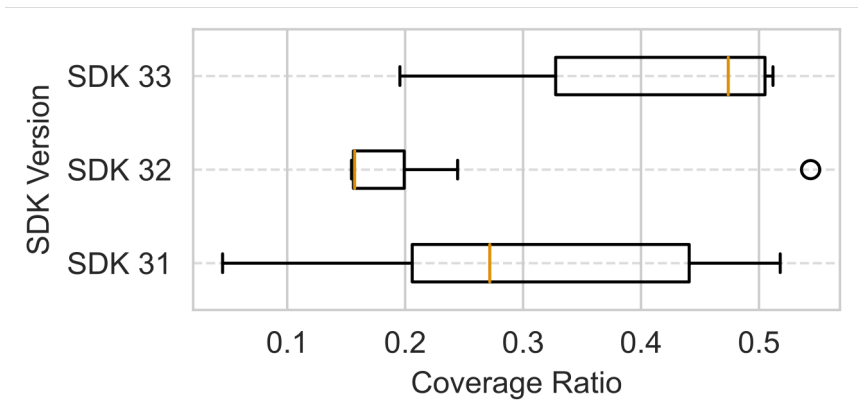
		Task Success				
Subset	#Sample	Build	Boot	Core	Launch	Avg. $S(\mathcal{D}_i)$
$\mathcal{D}_{33}$	73	1.0	1.0	0.41	0.41	0.70
$\mathcal{D}_{32}$	25	1.0	1.0	0.68	0.68	0.80
$\mathcal{D}_{31}$	86	0.9	0.9	0.18	0.18	0.54
Overall (Avg.)	184	0.97	0.97	0.42	0.42	0.68
Overall (Wtd. Avg.)	184	0.96	0.96	0.35	0.35	0.65

Re-hosting failures primarily occurred during core service initialization due to missing symbols, integrity checks, or limited emulator support.

Coverage

How much of the vendor filesystem is covered?

- The average coverage across SDK versions ranges from 19% to 42%, with the highest value observed for SDK 33.
 - Highly depended on samples and injection strategy

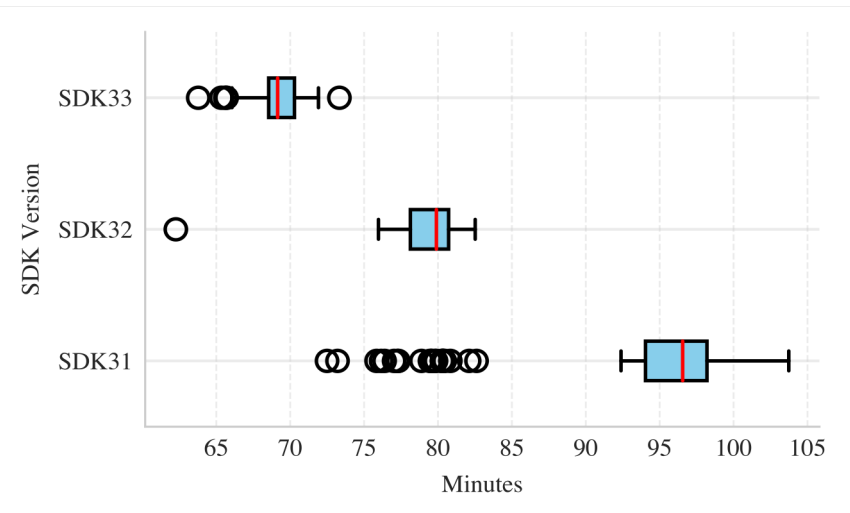


Baseline injection strategy is influenced by the SDK version and sample set.

Build Performance

What is the overhead of the matching and injection process?

- The pre- and post-injection stages add less than three minutes of overhead, confirming that injection performance costs are negligible for large-scale use.
- No significant impact on build procedure in terms of build failures and performance overhead.



Injection adds minimal overhead (<3min) and does not significantly slow down the AOSP build process.

Future Work

1. We are working on an extended journal version of the paper
 - More data samples and vendors
 - Testing pre-Installed apps runtime behaviour
 - Including a detailed failure analysis
2. End-to-End Dynamic Testing
 - Security Testing
 - Scaling Number of Data Samples from 184 to over 1k
3. Improving Injection Strategies
 - Vendor tuning
 - Device tuning
 - Improved Correctness and Coverage



Thanks for your attention!



Dataset & Source Code

Project Website: <https://sites.google.com/view/relocate-and-emulate>

Contact: thomas.sutter@unibe.ch | <https://7homassutter.github.io/>



Contact